Max Flow (Dinic)

```
#include<bits/stdc++.h>
#define N 209
#define M 20009
using namespace std;
#define rep(i,x,y) for(int i=(x);i<=(y);++i)
const int inf=0x3f3f3f3f;
int n,m,s,t,hd[N],tot,now[N],d[N],nV;
long long maxf;
struct edge{int t,w,nxt;} es[M<<1];
void adde(int u,int v,int
 ↪  w){es[++tot]=(edge){v,w,hd[u]},hd[u]=tot;es[++tot]=(edge){u,0,hd[v]},hd[v]=tot;}
bool bfs(){
    fill(d+1,d+nV+1,0);
    copy(hd+1,hd+nV+1,now+1);
    queue<int>q;q.push(s);d[s]=1;
    while(q.size()){
        int u=q.front();q.pop();
        for(int i=hd[u],v;i;i=es[i].nxt)if(!d[v=es[i].t]&&es[i].w){
            d[v]=d[u]+1;q.push(v);
            if(v==t)return 1;
        }
    }
    return 0;
}
int dinic(int u,int f){
    if(u==t)return f;
    int r=f;
    for(int i=now[u],v;i&&r;i=es[i].nxt)if(d[v=es[i].t]==d[u]+1&&es[now[u]=i].w){
        int k=dinic(v,min(r,es[i].w));
        if(!k)d[v]=0;
        es[i].w-=k,es[i^1].w+=k;r-=k;
    }
    return f-r;
}
int main(){
    cin>>nV>>m>>s>>t;tot=1;rep(i,1,m){int u,v,w;scanf("%d %d
 ↪  %d",&u,&v,&w);adde(u,v,w);}while(bfs())maxf+=dinic(s,inf);cout<<maxf<<endl;
    return 0;
}
```

Segment Tree Merge / Split

```
#include<bits/stdc++.h>
using namespace std;
#define N 200009
typedef long long ll;
#define rep(i,x,y) for(int i=(x);i<=(y);++i)
int n,m,a[N],rt[N],cnt,tot;
struct seg{int lc,rc;ll cnt;} tr[N*20];
#define lc(p) tr[p].lc
#define rc(p) tr[p].rc
#define cnt(p) tr[p].cnt
void New(int&p){p=++tot;}
void build(int p,int l,int r){
    if(l==r){cnt(p)=a[l];return;}
```

```
    int mid=l+r>>1;New(lc(p));build(lc(p),l,mid);New(rc(p));build(rc(p),mid+1,r);
    cnt(p)=cnt(lc(p))+cnt(rc(p));
}
int merge(int p,int q){
    if(!p||!q)return p|q;
    cnt(p)+=cnt(q);lc(p)=merge(lc(p),lc(q));rc(p)=merge(rc(p),rc(q));return p;
}
void split(int&p,int&q,ll k){
    if(!p)return;
    New(q);ll c=cnt(lc(p));
    if(k<=c){swap(rc(p),rc(q));split(lc(p),lc(q),k);}else split(rc(p),rc(q),k-c);
    cnt(q)=cnt(p)-k;cnt(p)=k;
}
ll query(int p,int l,int r,int ql,int qr){
    if(!p||ql<=l&&r<=qr)return cnt(p);
    int mid=l+r>>1;ll
     ↪ ans=0;if(ql<=mid)ans+=query(lc(p),l,mid,ql,qr);if(qr>mid)ans+=query(rc(p),mid+1,r,ql,qr);
    return ans;
}
int kth(int p,int l,int r,ll k){
    if(k>cnt(p))return -1;if(l==r)return l;
    int mid=l+r>>1;ll c=cnt(lc(p));if(k<=c){if(!lc(p))New(lc(p));return kth(lc(p),l,mid,k);}else
     ↪ {if(!rc(p))New(rc(p));return kth(rc(p),mid+1,r,k-c);}
}
void insert(int p,int l,int r,int x,int c){
    cnt(p)+=c;if(l==r)return;
    int mid=l+r>>1;
    if(x<=mid){if(!lc(p))New(lc(p));insert(lc(p),l,mid,x,c);}
    else{if(!rc(p))New(rc(p));insert(rc(p),mid+1,r,x,c);}
}
int main(){
    cin>>n>>m;rep(i,1,n)scanf("%d",&a[i]);
    New(rt[++cnt]);build(rt[1],1,n);
    while(m--){
        int op,t,p,x,y;scanf("%d",&op);
        if(!op){
            scanf("%d %d %d",&p,&x,&y);
            ll k1=query(rt[p],1,n,1,y),k2=query(rt[p],1,n,x,y);
            split(rt[p],t,k1);split(rt[p],rt[++cnt],k1-k2);
            rt[p]=merge(rt[p],t);
        }else if(op==1){scanf("%d %d",&p,&t);rt[p]=merge(rt[p],rt[t]);}
        else if(op==2){scanf("%d %d %d",&p,&x,&y);insert(rt[p],1,n,y,x);}
        else if(op==3){scanf("%d %d %d",&p,&x,&y);printf("%lld\n",query(rt[p],1,n,x,y));}
        else{scanf("%d %d",&p,&t);printf("%d\n",kth(rt[p],1,n,t));}
    }
    return 0;
}
```